Operating Systems Note

Contents

1 Introduction **1**
- 1.1 Execution Model . . . 1
- 1.2 Main Memory vs. Hard Disk . . . 1

2 x86 Privilege Rings **2**

3 Interrupts **2**
- 3.1 Basics . . . 2
- 3.2 What Happens on an Interrupt . . . 2
- 3.3 Why Interrupts Matter . . . 3
- 3.4 Asynchronous vs. Synchronous Interrupts . . . 3
- 3.5 System Calls . . . 3

4 Processes **4**
- 4.1 Program vs. Process . . . 4
- 4.2 Process Control Block (PCB) . . . 4
- 4.3 Parent and Child Processes . . . 4
- 4.4 Process States and Context Switching . . . 4

5 Number Representation (Endianness) **5**
- 5.1 Example . . . 5

6 Function Calls and the Stack **6**
- 6.1 Saving Registers . . . 6
- 6.2 Example: One-Register Machine . . . 6
- 6.3 Arguments and Return Values . . . 6

7 Booting **7**
- 7.1 Power-On . . . 7
- 7.2 Boot Sequence . . . 7
- 7.3 Master Boot Record (MBR) . . . 8
- 7.4 Kernel Initialization . . . 8
- 7.5 Init / Systemd . . . 9

A Glossary **9**

1 Introduction

1.1 Execution Model

- **Assumption:** a machine with a single-core CPU, one program counter (`PC`) and one instruction register (`IR`).
- The CPU repeatedly performs the *fetch–execute* cycle: `PC` holds the address of the next instruction, that instruction is fetched into `IR`, and then it is executed.
- When an application is running, the CPU executes the application's instructions directly on the hardware, not on top of the OS. This is called running on **bare metal**.
- The OS is *not* executing while an application runs. It only gets the CPU back through an interrupt or a system call (Section 3).
- So, for native (non-interpreted) code, whatever is currently running is running on bare metal.

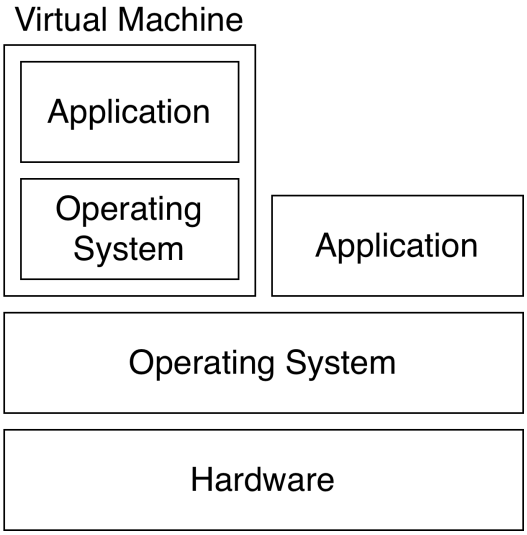


Figure 1: The usual layered depiction

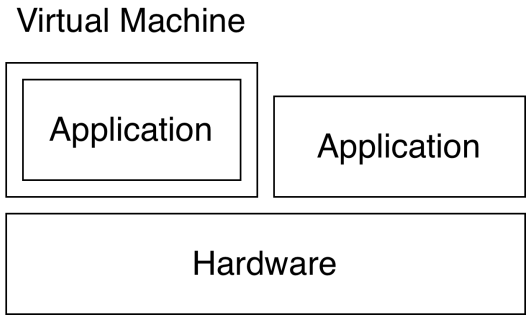


Figure 2: What is actually happening: the running code executes directly on the hardware

Interpreters and virtual machines

Code run by an interpreter (e.g. Python, Java bytecode in a JVM) does not itself run on bare metal. The interpreter does, and it executes the code on its behalf. With hardware-assisted virtualization, a guest OS's instructions typically also run directly on the CPU, with a hypervisor stepping in only when needed.

1.2 Main Memory vs. Hard Disk

- Main memory is **byte addressable**.
- A hard disk is accessed in blocks called **sectors**. Positioning on a disk (seek and rotation) is slow compared to memory, so transferring one byte at a time would waste CPU time. Data is therefore moved in fixed-size chunks.
- Both are typically drawn as an array of slots for ease of understanding.

	Main memory (RAM)	**Hard disk**
Smallest access unit	1 byte	1 sector (commonly 512 B)
Relative speed	Fast	Slow
Volatile?	Yes	No

2 x86 Privilege Rings

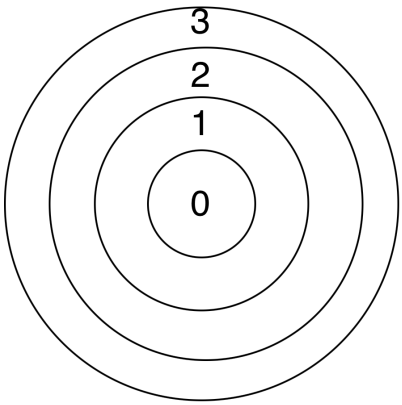


Figure 3: x86 protection rings

- The rings are privilege levels: **0 is the most privileged, 3 the least**.
- The CPU uses them to restrict which memory can be accessed and which instructions can be executed, so that buggy or malicious software cannot crash the system.
- **Ring 0 – Kernel mode:** the OS kernel runs here with unrestricted access to memory and hardware.
- **Rings 1 & 2:** intended for device drivers and OS services needing intermediate privilege.
- **Ring 3 – User mode:** applications and user-space processes run here with restricted hardware access and isolated virtual memory.

- The current ring is kept by the CPU itself (the *current privilege level*).
- Code cannot simply set itself to ring 0. The only ways to gain privilege are the controlled entry points: **interrupts, exceptions and system calls** (Section 3). The CPU then jumps to OS code chosen by the OS, never to an address chosen by the application.

In practice

Mainstream operating systems (Windows, Linux) use only **ring 0 and ring 3**. Rings 1 and 2 exist in the architecture but are essentially unused.

3 Interrupts

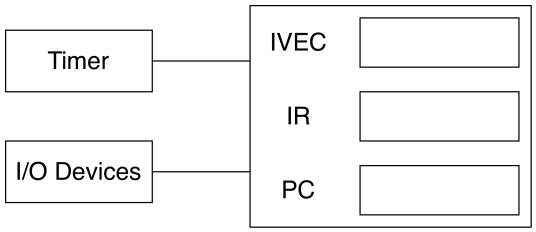


Figure 4: Devices connect to the CPU through interrupt lines

3.1 Basics

- Interrupts are signals generated by I/O devices (keyboard, mouse, ...) or by the timer. These devices are connected to the CPU through interrupt lines.
- The `IVEC` (interrupt vector) register holds the memory address of the first instruction of the interrupt handler. The handler is part of the OS.
- After the handler finishes, execution must return to where it was interrupted. So the old `PC` is saved on the stack before the handler starts, exactly like the return address of a function call (Section 6).

3.2 What Happens on an Interrupt

1. An application is running; the CPU is fetching its instructions.
2. An interrupt signal arrives on an interrupt line. The CPU finishes the instruction it is executing.
3. The old `PC` (and flags) are saved on the stack.
4. The ring level is set to 0, so the handler runs with the highest privilege.

5. The value in `IVEC` is copied into `PC`.
6. The next fetch reads the handler's first instruction (from the OS's memory) into `IR`.
7. The handler runs, so the OS is now running.
8. At the end, a special return-from-interrupt instruction (`iret` on x86) restores the saved `PC` and the previous ring, and the application resumes where it was interrupted.

The exact order of the hardware steps differs between CPUs; what matters is that the old state is saved, privilege is raised, and `PC` is redirected to the handler.

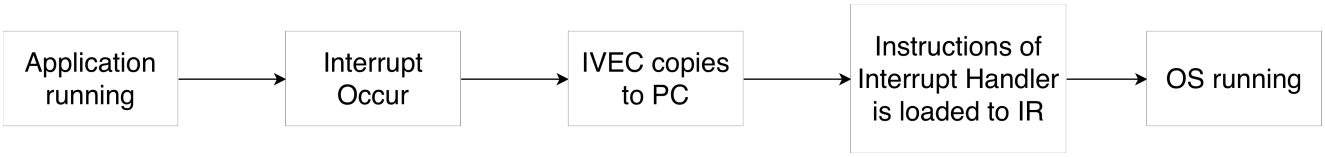


Figure 5: Control transfer from application to OS on an interrupt (the return step is not shown)

Common misconception

The interrupt changes `PC`, not `IR`. `IR` merely holds the instruction currently being executed; it receives the handler's first instruction on the next fetch.

3.3 Why Interrupts Matter

- Without interrupts, an application could run forever and hog the CPU.
- Keyboard and mouse interrupts happen only when the user acts, so they cannot be relied on. The **timer** therefore raises an interrupt at regular intervals, guaranteeing that the OS regains control periodically.

3.4 Asynchronous vs. Synchronous Interrupts

	Asynchronous	**Synchronous**
Caused by	External hardware (timer, keyboard, disk)	The running program's own instruction
Timing	Can strike at any point; the program cannot predict it	Happens at a known instruction
Program aware?	No, it does not necessarily know it happened	Yes
Examples	Timer tick, key press, disk finished	System call (`int`/`syscall`), divide by zero, page fault
Returns to	The instruction that was about to run	Usually the next instruction (system call); the same instruction for a fault that is fixed and retried

3.5 System Calls

- Privileged services (writing to the screen, disk or network) need ring 0, but applications run in ring 3. An application therefore asks the OS to do the work by raising a **synchronous interrupt** on purpose. This is a **system call**.
- On x86 this is done with the `int` instruction (e.g. `int 0x80` on 32-bit Linux) or the faster `syscall` instruction on x86-64. The idea is the same.

1. The program calls a library function such as `write()`. This function is a **wrapper**: ordinary user-space code that does no real work itself.
2. The wrapper puts the **system call number** (and the arguments) in registers and executes `int`/`syscall`.
3. The CPU switches to ring 0 and jumps to the interrupt handler, as in Section 3.
4. The handler reads the system call number from the register and uses it to look up the matching kernel service (a table of function pointers).

5. The kernel carries out the service on behalf of the process.
6. The handler returns to user mode (`iret`/`sysret`), the wrapper returns, and the program continues.

Wrapper vs. handler

The *wrapper* (e.g. `write()` in the C library) is user code that triggers the interrupt. The privileged work is done by the kernel's implementation of the system call, to which the interrupt handler dispatches.

Which stack does the handler use?

When an interrupt moves the CPU from ring 3 to ring 0, the CPU switches to a **kernel stack** (each process has one), because the user stack cannot be trusted: it may be too small, corrupted or deliberately malicious. A simplified model may say the handler uses the interrupted program's stack. When an interrupt occurs while the kernel is already running, no switch is needed.

Real hardware

A single `IVEC` register is a simplification. On x86 there are many interrupt sources, each with its own handler, so the CPU uses an *interrupt vector table* (the IDT in protected mode), which holds one handler address per interrupt number.

4 Processes

4.1 Program vs. Process

- A **program** is a passive file on disk: instructions and data.
- A **process** is a program *in execution*.
- A process is more than its instructions. To understand it fully you need its **state**: its code, its memory (data, heap, stack), the values of the registers including `PC`, and the files it has open. Two processes can run the same program with different state.
- Each process has its own address space and its own stack in memory.

4.2 Process Control Block (PCB)

The OS keeps one **PCB** per process, a kernel data structure that records everything it needs to manage the process and to resume it later.

PCB field	Purpose
Process ID (PID), parent PID	Identify the process and its parent
State	Ready, running, blocked, ...
Saved `PC` and registers	Restore the CPU exactly as the process left it
Memory information	Where the process's memory is
Open files	Files and devices the process is using

4.3 Parent and Child Processes

- New processes are created by an existing process (on Linux with `fork()`, often followed by `exec()`). The creator is the **parent**.
- Every process has a parent, except the very first one (`init`, PID 1), which the kernel starts itself (Section 7.5).
- If a parent terminates first, its children become **orphans** and are adopted by `init`.

4.4 Process States and Context Switching

- The CPU never stops fetching and executing instructions while the machine is on, so there must always be something for it to run.
- A process that must wait for an event (e.g. a disk read) cannot continue, so it becomes **blocked**. The OS then runs another process that is **ready**.

- If no process is ready, the OS runs the **idle process**, which does nothing useful (conceptually `while(1);` real kernels execute a halt instruction to save power) until an interrupt brings new work.
- The timer interrupt also limits how long a process may run. When its time is up, it goes back to the ready queue and another process gets the CPU.

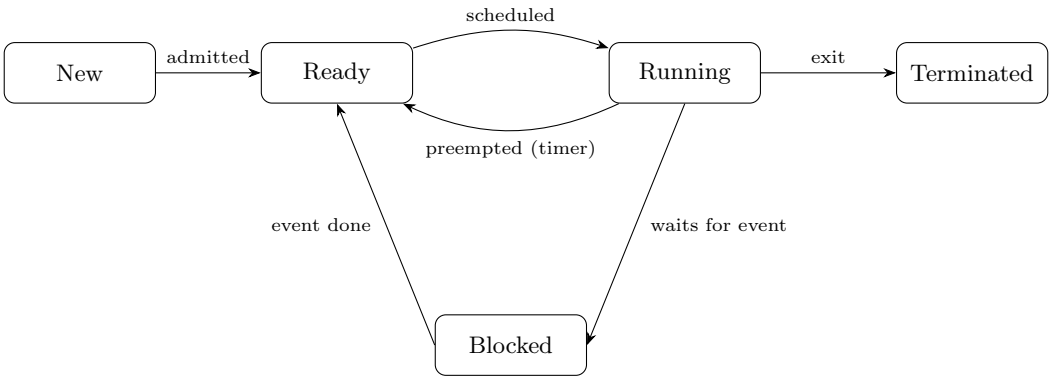


Figure 6: Process states

Context switch. Moving the CPU from one process to another:

1. An interrupt (timer or I/O) or a system call hands control to the OS.
2. The OS saves the running process's `PC` and registers into its PCB.
3. The scheduler picks the next ready process.
4. The OS loads that process's saved `PC` and registers from its PCB and returns to it.

Common misconception

The OS does not "ignore" a waiting process or throw away its return address. All of its state is saved in the PCB, so the process can later continue exactly where it stopped.

5 Number Representation (Endianness)

- A number that spans multiple bytes has a **Most Significant Byte (MSB)** and a **Least Significant Byte (LSB)**.
- *Endianness* defines the order in which these bytes are stored in memory.
- **Big endian:** the MSB is stored at the *lowest* address.
- **Little endian:** the LSB is stored at the *lowest* address.
- x86 uses **little endian**.

5.1 Example

Consider the 4-byte number `0x12345678`, with MSB = `12` and LSB = `78`.

Address	`+0`	`+1`	`+2`	`+3`
Big endian	`12`	`34`	`56`	`78`
Little endian	`78`	`56`	`34`	`12`

Why it matters later

Endianness affects multi-byte fields on disk too. For example, the MBR signature (Section 7.3) is the byte sequence `55 AA`, which reads as the 16-bit value `0xAA55` on a little-endian machine.

6 Function Calls and the Stack

6.1 Saving Registers

- When a function is going to use a register that already holds a value the caller needs, the compiler emits instructions that `push` the register onto the stack at the start of the function and `pop` it back at the end.
- This preserves the caller's values.
- The exact rules (whether caller or callee saves which registers) are defined by the **calling convention**.

Common misconception

Declaring a variable does not by itself push a register. Local variables are either kept in registers or allocated in the function's stack frame (by adjusting the stack pointer). Pushes happen to *save registers that the function will overwrite*.

6.2 Example: One-Register Machine

```c
#include <stdio.h>

void func() {
        int x = 0;
        x++;
        printf("%d\n", x);
}

int main() {
        int b = 10;
        func();
        printf("%d\n", b);
        return 0;
}
```

The output is `1` followed by `10`.

Assume the machine has only one register, `R`, for holding variable values.

1. `main` runs first and stores `b = 10` in `R`.
2. The call instruction pushes the **return address** onto the stack, then sets `PC` to the address of the first instruction of `func`.
3. At the start of `func`, before `R` is reused for `x`, the old value of `R` (10) is pushed onto the stack.
4. `func` sets `R` to 0, increments it to 1 and prints it.
5. Before returning, `func` pops the saved value back into `R`, so `R` holds 10 again.
6. The return instruction pops the return address into `PC`, and `main` resumes where it left off and prints 10.

Stack (top first)	State
`saved R = 10`	after the push inside `func`
`return address`	pushed by the call

Values are popped in the reverse order they were pushed (LIFO): the saved register first, then the return address.

6.3 Arguments and Return Values

The calling convention also says how arguments and the result are passed. On **32-bit x86** (the `cdecl` convention) the caller pushes the arguments on the stack, and the result comes back in a register (`eax`). On x86-64 the first few arguments are passed in registers instead, and the stack is used for extra ones.

```c
#include <stdio.h>

int add(int a, int b) {
        return a + b;
}

int main() {
        int r = add(2, 3);
        printf("%d\n", r);
        return 0;
}
```

1. The caller pushes the arguments, last one first: `3`, then `2`.
2. The call instruction pushes the return address and sets `PC` to `add`.
3. `add` reads `a` and `b` from fixed positions relative to the stack pointer, and leaves the sum in `eax`.
4. The return instruction pops the return address into `PC`.
5. The caller removes the two arguments from the stack and copies `eax` into `r`.

Stack (top first)	Pushed by
`return address`	the call instruction
`a = 2`	the caller
`b = 3`	the caller

7 Booting

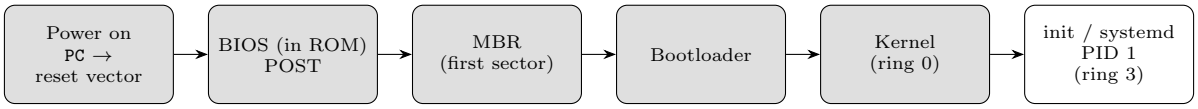


Figure 7: Boot chain. Shaded stages run with full privilege.

7.1 Power-On

- RAM contents are undefined at power-on. Most registers also hold arbitrary values, but the hardware resets a few (including `PC`) to defined values.
- `PC` is set to a fixed **reset vector** in ROM. ROM is non-volatile, so unlike RAM its contents survive power loss.
- On classic x86 the reset vector is 16 bytes below the top of the address space. These 16 bytes hold a jump to the entry code of the **BIOS** firmware. This indirection allows different machines to have different ROM sizes and layouts.
- The BIOS runs in real mode with unrestricted hardware access (the privilege rings do not apply yet).

7.2 Boot Sequence

1. **POST** (Power-On Self Test): the BIOS checks that the hardware is present and working.
2. The BIOS reads the **first sector** of the boot disk into RAM. This is the MBR (Section 7.3).
3. The BIOS jumps to the start of that code in RAM (the *boot call*).
4. The MBR code loads a **bootloader** (directly, or via a second stage or the active partition's boot sector).
5. The bootloader loads the **OS kernel** into memory and jumps to it.
6. The kernel initializes the system (Section 7.4).
7. The kernel starts the first user process, `init` (Section 7.5).

UEFI

Modern machines use UEFI firmware instead of the legacy BIOS and typically a GPT partition scheme instead of the MBR. The overall idea (firmware → bootloader → kernel) is the same.

7.3 Master Boot Record (MBR)

Consider a disk with 512-byte sectors. The MBR is the first sector.

Offset	Size	Content
0	446 bytes	Boot code
446	64 bytes	Partition table (4 entries × 16 bytes)
510	2 bytes	Boot signature: `55` at 510, `AA` at 511 (written 0x55AA)

- The signature marks the sector as bootable. Read as a little-endian 16-bit word it is `0xAA55`.
- The table holds up to **4 primary partitions**.

Field	Size	Meaning
Status	1 B	`0x80` = active (bootable), `0x00` = inactive
Start CHS	3 B	Start address in cylinder/head/sector form
Type	1 B	Partition type code
End CHS	3 B	End address in CHS form
Start LBA	4 B	First sector, as a logical block address
Sector count	4 B	Size of the partition in sectors

Partition entry (16 bytes).

- There is no stored partition number; it is implied by the entry's position in the table.
- The type is a *code*, not a filesystem name. For example `0x07` is NTFS/exFAT and `0x83` is Linux (used for ext4). A code can map to several filesystems.

Magic numbers

A **magic number** is a fixed byte pattern at the start of a file that identifies its format. On Linux, the kernel uses it to decide how to run an executable, and the `file` command uses it (plus other heuristics) to identify files. The file extension does not matter.

- ELF executables: `0x7F` followed by `ELF`
- Java class files: `0xCAFEBABE`
- Scripts: `#!` (the magic) followed by an interpreter path, e.g. `#!/bin/bash`. The path can be any interpreter.

7.4 Kernel Initialization

Once the bootloader hands over control, the kernel (running in ring 0) prepares the machine for use:

- It sets up its own interrupt handling: it fills in `IVEC` (the interrupt vector table) and configures the **timer** so that it regains control periodically.
- It initializes memory management, the scheduler and the device drivers.
- It **mounts** the root file system, so that files can be read.
- Finally it starts the first user process, `init`.

Common misconception

Creating file systems (formatting) and configuring the network are not part of the kernel's boot work. Mounting the root file system is. Network and most service configuration is done later by user-space programs started by `init`/`systemd`.

7.5 Init / Systemd

- After the kernel has finished initializing, the first *user-space* process it starts is `init` (or `systemd` on newer Linux systems).
- The CPU is now switched to **ring 3**, the lowest privilege level, for this process.
- Since it is the first process, it has **PID 1**.
- It is the only user-space process the kernel starts directly. `init` then starts all the others: the classic SysV `init` reads `/etc/inittab`, while `systemd` reads its unit files (e.g. under `/etc/systemd/system`).
- It is also the adoptive parent of orphaned processes (Section 4).

Root is not ring 0

Processes started by `init` may run as the **root** user (superuser), but they still execute in **ring 3**. Root is an *OS-level permission* (what the kernel allows), whereas the ring is a *CPU-level privilege* (what the hardware allows). A root process still needs system calls to reach the hardware.

Kernel threads

The kernel also creates its own threads that run entirely in kernel mode, e.g. the idle task (PID 0) and `kthreadd` (PID 2). They are not user-space processes, which is why `init` is the only *user* process started by the kernel.

A Glossary

Term	Meaning
`PC`	Program counter: address of the next instruction
`IR`	Instruction register: the instruction being executed
`IVEC`	Interrupt vector register: address of the interrupt handler
Ring	CPU privilege level (0 = most privileged, 3 = least)
Interrupt handler	OS code that runs when an interrupt occurs
System call	A deliberate request from a program to the OS, made via a synchronous interrupt
Wrapper	User-space library function (e.g. `write()`) that issues a system call
Process	A program in execution
PCB	Process control block: the kernel's record of a process
PID	Process ID
Context switch	Saving one process's state and loading another's
Idle process	Process run when nothing else is ready
BIOS	Firmware in ROM that starts the boot process
POST	Power-On Self Test, run by the BIOS
MBR	Master Boot Record: first sector of a disk
Bootloader	Program that loads the kernel into memory
`init` / `systemd`	First user-space process (PID 1)
MSB / LSB	Most / least significant byte
LIFO	Last in, first out (stack order)